6: Kubernetes Deployments

In this chapter, you'll see how to use *Deployments* to bring cloud-native features such as self-healing, scaling, rolling updates, and versioned rollbacks to stateless apps on Kubernetes.

The chapter is divided as follows:

- Deployment theory
- Create a Deployment
- Perform scaling operations
- Perform a rolling update
- Perform a rollback

Kubernetes offers several *controllers* that augment Pods with important capabilities. The Deployment controller is specifically designed for stateless apps. We'll cover some other controllers later in the book.

Deployment theory

There are two major pieces to Deployments.

1. The spec
2. The controller

The *Deployment spec* is a declarative YAML object where you describe the desired state of a stateless app. You give that to Kubernetes where the *Deployment controller* implements and manages it. The controller aspect is highly-available and operates as a background loop reconciling observed state with desired state.

Deployment objects, and all of their features and attributes, are defined in the `apps/v1` workloads API sub-group.

> **Note**: The Kubernetes API is architecturally divided into smaller sub-groups to make it easier to manage and navigate. The `apps` sub-group is where Deployments, DaemonSets, StatefulSets, and other *workload*-related objects are defined. We sometimes call it the workloads API.

You start with a stateless application, package it as a container, then define it in a Pod template. At this point you *could* run it on Kubernetes. However, static Pods like this don't self-heal, they don't scale, and they don't allow for easy updates and rollbacks. For these reasons, you'll almost always wrap them in a Deployment object.

Figure 6.1 shows a Pod template wrapped in a Deployment object. In fact, there are three levels of nesting. The container holds the application, the Pod augments the container with labels, annotations, and other metadata useful for Kubernetes, and the Deployment further augments things with scaling and updates.

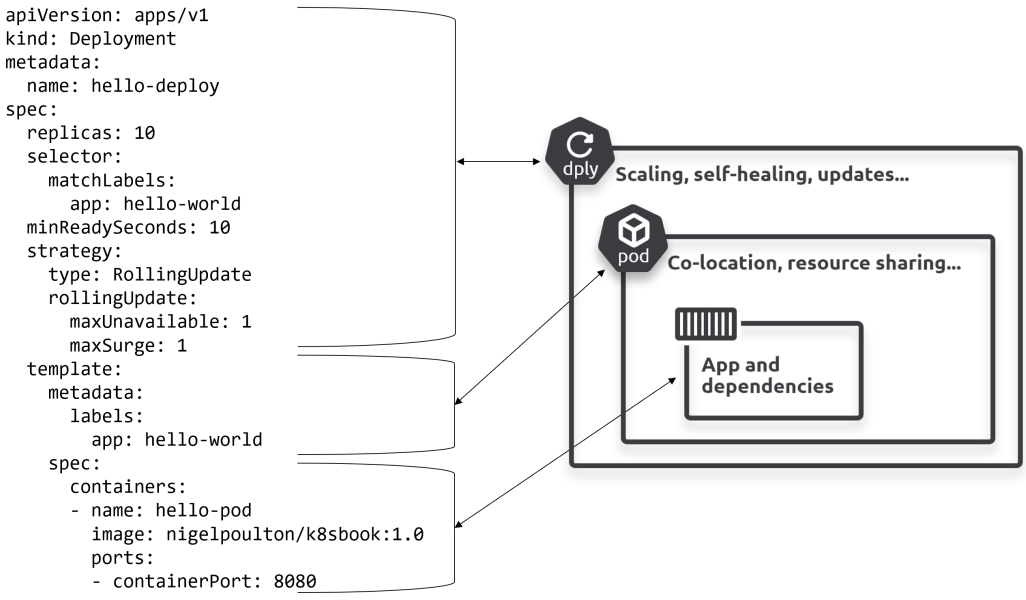

Figure 6.1

You post the Deployment object to the API server where, Kubernetes implements it and the Deployment controller watches it.

Deployments and Pods

A Deployment object only manages a single Pod template. For example, an application with a front-end web service and a back-end catalog will have a different Pod for each (two Pod templates). As a result, it'll need two Deployment objects – one managing front-end Pods, the other managing back-end Pods. However, a Deployments can manage multiple replicas of the same Pod. For example, the front-end Deployment might be managing 5 identical front-end Pod replicas.

Deployments and ReplicaSets

Behind-the-scenes, Deployments rely heavily on another object called a ReplicaSet. While it's usually recommended not to manage ReplicaSets directly (let the Deployment controller manage them), it's important to understand the role they play.

At a high-level, containers are a great way to package applications and dependencies. Pods allow containers to run on Kubernetes and enable co-scheduling and a bunch of other good stuff. ReplicaSets manage Pods and bring self-healing and scaling. Deployments manage ReplicaSets and add rollouts and rollbacks.

Figure 6.2. is similar to 6.1, but adds a ReplicaSet into the relationship and shows which object is responsible for which features.

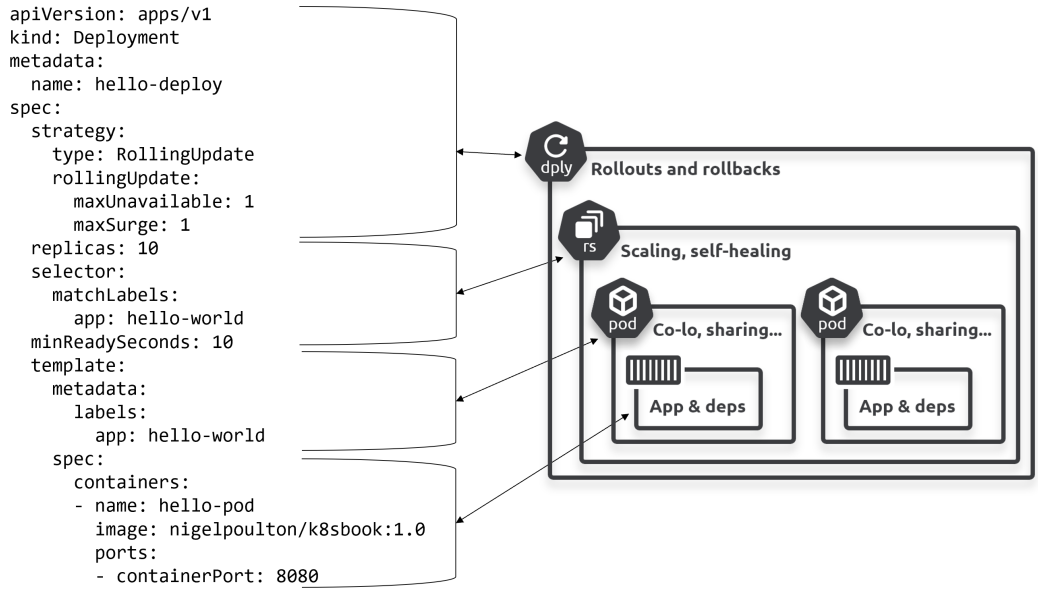

Figure 6.2

Think of *Deployments* as managing *ReplicaSets*, and *ReplicaSets* as managing *Pods*. Put them together, and you've got a great way to deploy and manage stateless applications on Kubernetes.

Self-healing and scalability

Pods are great. They let you co-locate containers, share volumes, share memory, simplify networking, and a lot more. But they offer nothing in the way of self-healing and scalability – if the node a Pod is running on fails, the Pod is lost.

Enter Deployments…

- If a Pod managed by a Deployment fails, it will be replaced – *self-healing*
- If Pods managed by a Deployment see increased or decreased load, they can be *scaled*

Remember though, behind-the-scenes, it's actually the ReplicaSets doing the self-healing and scalability. You'll see them in action soon.

It's all about the *state*

Before going any further, it's critical to understand three concepts that are fundamental to everything about Kubernetes:

- Desired state
- Observed state (sometimes called *actual state* or *current state*)

- Reconciliation

Desired state is what you **want**. *Observed state* is what you **have**. If they match, everybody's happy. If they don't match, a process of *reconciliation* brings them back together.

The *declarative model* is a method for telling Kubernetes your *desired state*, while avoiding the detail of *how* to implement it. You leave the *how* up to Kubernetes.

Declarative vs imperative

There are two competing models. The *declarative model* and the *imperative model*.

The declarative model is all about describing an end-goal – telling Kubernetes *what* you want. The imperative model is all about long lists of commands to reach an end-goal – telling Kubernetes *how* to do something.

The following is an extremely simple analogy that might help:

- **Declarative**: Make me a chocolate cake to feed 10 people.
- **Imperative**: Drive to store. Buy; eggs, milk, flour, cocoa powder… Drive home. Pre-heat oven. Mix ingredients. Place in baking tray. Place tray in oven for 30 minutes. Remove from oven and turn oven off. Add icing. Leave to stand.

The declarative model is stating what you want (chocolate cake for 10). The imperative model is a long list of steps that will *hopefully* make a chocolate cake for 10.

Let's look at a more concrete example.

Assume an application with two microservices – front-end and back-end. To meet expected demand, you know you need 5 instances of the front-end Pod and 2 instances of the back-end Pod.

Taking the declarative approach, you write a simple configuration file telling Kubernetes you want 5 replicas of the front-end Pod all listening externally on port 80. You also want 2 back-end Pods listening internally on port 27017. That's your desired state.

Once you've described this in a config file, you give it to Kubernetes and sit back while Kubernetes does the hard work of implementing and monitoring it. It's a beautiful thing.

The opposite of this is the imperative model. It has no concept of desired state, it's just a list of steps and instructions.

To make things worse, imperative instructions can have endless potential variations. For example, the commands to pull and start `containerd` containers are different from the commands to pull and start `cri-o` containers. This creates more work, is prone to more errors, and because it's not declaring a desired state, there's no self-healing. Believe me when I tell you, this is not so beautiful.

Kubernetes supports both models, but strongly prefers the declarative model.

> **Note**: *containerd* and *cri-o* are CRI container runtimes that Kubernetes runs on cluster nodes instead of Docker to perform low-level tasks such as starting and stopping containers.

Controllers and reconciliation

Fundamental to *desired state* is the process of *reconciliation.*

For example, ReplicaSets are implemented as a controller running as a background reconciliation loop checking the right number of Pod replicas are present on the cluster. If there aren't enough, it adds more. If there are too many, it terminates some.

Assume a scenario where desired state is 10 replicas, but only 8 are present. It makes no difference if this is due to a failure, or if it's because an autoscaler has increased desired state from 8 to 10. Either way, this is a red-alert condition for Kubernetes, so it orders the control plane to general quarters and brings up two more replicas. And the best part… it does all this without calling you at 4:20 am!

The exact same reconciliation process powers self-healing, scaling, rollouts, and rollbacks. Let's take a closer look.

Rolling updates with Deployments

Zero-downtime rolling-updates of stateless apps are what Deployments are all about, and they're amazing. However, they require a couple of things from your microservices applications in order to work properly.

1. Loose coupling via APIs
2. Backwards and forwards compatibility

Both of these are hallmarks of modern cloud-native microservices apps and work as follows.

All microservices in an app should be decoupled and only communicate via well-defined APIs. This allows any microservice to be updated without having to think about clients and other microservices that interact with them – everything talks to formalised APIs that expose documented interfaces and hide specifics. Ensuring releases are backwards and forwards compatible means you can perform independent updates without having to factor in which versions of clients are consuming the service.

With those points in mind, zero-downtime rollouts work like this.

Assume you're running 5 replicas of a stateless web front-end. As long as all clients communicate via APIs and are backwards and forwards compatible, it doesn't matter which of the 5 replicas a client connects to. To perform a rollout, Kubernetes creates a new replica running the new version and terminates an existing one running the old version. At this point, you've got 4 replicas on the old version and 1 on the new. This process repeats until all 5 replicas are on the new version. As the app is stateless, and there are always multiple replicas up and running, clients experience no downtime or interruption of service.

There's actually a lot that goes on behind the scenes, so let's look a bit closer.

You design applications with each discrete microservice as its own Pod. For convenience – self-healing, scaling, rolling updates and more – you wrap the Pods in their own higher-level controller such as a Deployment. Each Deployment describes all the following:

- How many Pod replicas
- What images to use for the Pod's container(s)
- What network ports to expose
- Details about how to perform rolling updates

In the case of Deployments, when you post the YAML file to the API server, the Pods get scheduled to healthy nodes and a Deployment and ReplicaSet work together to make the magic happen. The ReplicaSet controller sits in a watch loop making sure our old friends *observed state* and *desired state* are in agreement. A Deployment object sits above the ReplicaSet, governing its configuration, as well as how rollouts will be performed.

All good so far.

Now, assume you're exposed to a known vulnerability and need to rollout a newer image with the fix. To do this, you update the **same Deployment YAML file** with the new image version and re-post it to the API server. This updates the existing Deployment object with a new desired state requesting the same number of Pods but all running the newer image.

To make this happen, Kubernetes creates a second ReplicaSet to create and manage the Pods with the new image. You now have two ReplicaSets – the original one for the Pods with the old image, and a new one for the Pods with the new image. As Kubernetes increases the number of Pods in the new ReplicaSet (with the new version of the image) it decreases the number of Pods in the old ReplicaSet (with the old version of the image). Net result, you get a smooth incremental rollout with zero downtime.

You can rinse and repeat the process for future updates – just keep updating the same Deployment manifest, which should be stored in a version control system.

Brilliant.

Figure 6.3 shows a Deployment that's been updated once. The initial release created the ReplicaSet on the left, and the update created the one on the right. You can see the ReplicaSet for the initial release has been wound down and no longer manages any Pods. The one associated with the update is active and owns all the Pods.

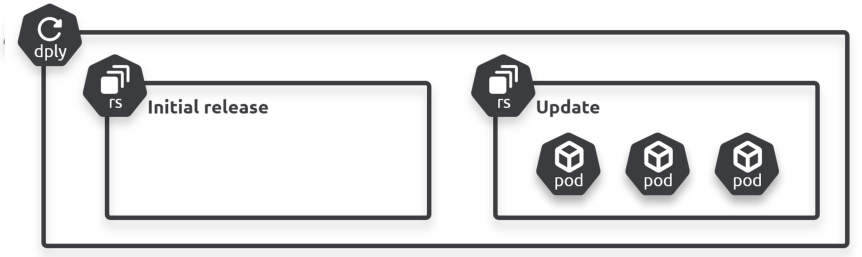

Figure 6.3

It's important that the old ReplicaSet from the initial release still exists with its configuration intact. You'll see why in the next section.

Rollbacks

As you saw in Figure 6.3, older ReplicaSets are wound down and no longer manage any Pods. However, their configurations still exist on the cluster, making them a great option for reverting to previous versions.

The process of a rollback is the opposite of a rollout – you wind one of the old ReplicaSets up while you wind the current one down. Simple.

Figure 6.4 shows the same app rolled back to the initial release.

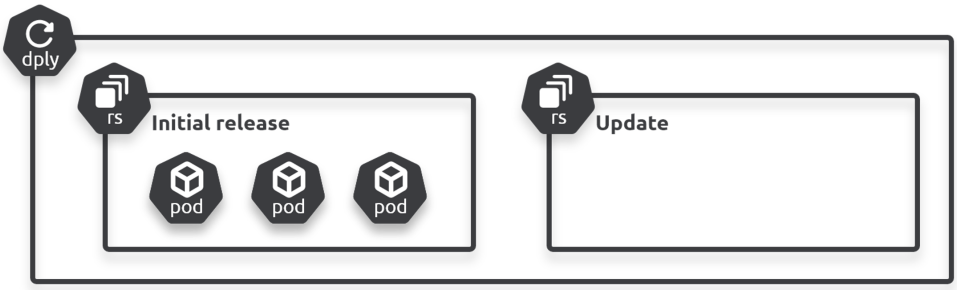

Figure 6.4

That's not the end though. Kubernetes gives you fine-grained control over how updates and rollbacks proceed. For example, you can insert delays, control the pace and cadence of the release, and you can even probe the health and status of updated replicas.

But talk is cheap, let's see this stuff in action.

Create a Deployment

You'll need a Kubernetes cluster and the lab files from the book's GitHub repo if you want to follow along.

```
$ git clone https://github.com/nigelpoulton/TheK8sBook.git
Cloning into 'TheK8sBook'...
```

If you don't know how to use `git`, or can't install it, you can just go to the GitHub repo and copy the text from the relevant files into files with the same name on your local machine.

Be sure to run all commands from within the `deployments` folder.

```
$ cd TheK8sBook/deployments
```

The following YAML snippet is from the `deploy.yml` file. It defines a single-container Pod wrapped in a Deployment object. It's been annotated to highlight some important sections.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy       <<==== Deployment name. Must be valid DNS name
spec:
  replicas: 10             <<==== Number of Pods to deploy & manage
  selector:
    matchLabels:
      app: hello-world
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 300
  minReadySeconds: 10
```

```
  strategy:                <<==== This block controls how updates happen
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:                <<==== Below here is the PodTemplate
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
      - name: hello-pod
        image: nigelpoulton/k8sbook:1.0
        ports:
        - containerPort: 8080
```

Let's step through it and explain some of the important parts.

Right at the very top, you specify the API version to use. At the time of writing, the latest stable Deployment schema is defined in the `apps/v1` API sub-group.

Next, the `kind` field tells Kubernetes you're defining a Deployment object.

The `metadata` section gives the Deployment a name. This should be a valid DNS name. So, alphanumerics, the dot and the dash are valid, and you should avoid exotic characters.

The `spec` section is where most of the action happens. Anything directly below `.spec` relates to the Deployment. Anything nested below `.spec.template` is the Pod template the Deployment uses to stamp out Pod replicas. In this example, the Pod template defines a single-container Pod.

`spec.replicas` is how many Pod replicas the Deployment should create and manage.

`spec.selector` is a list of labels that Pods must have in order for the Deployment to manage them. Notice how the Deployment selector matches the labels assigned to the Pod lower down in the Pod template (`app=hello-world`).

`spec.revisionHistoryLimit` tells Kubernetes how many older versions/ReplicaSets to keep. Keeping more gives you more rollback options, but keeping too many can bloat the object. This can be a problem on large clusters with lots of software releases.

`spec.progressDeadlineSeconds` tells Kubernetes how long to wait during a rollout for each new replica to come online. The example sets a 5 minute deadline, meaning each new replica has 5 minutes to come up before Kubernetes considers the rollout stalled. To be clear, the clock is reset after each new replica comes up, meaning each step in the rollout gets its own 5 minute window.

And finally, `spec.strategy` tells the Deployment controller how to update the Pods when a rollout occurs.

Use `kubectl apply` to deploy it on the cluster.

> **Note:** `kubectl apply` posts the YAML file to the Kubernetes API server and includes all necessary authentication tokens.

```
$ kubectl apply -f deploy.yml
deployment.apps/hello-deploy created
```

At this point, the Deployment configuration is persisted to the cluster store as a record of intent and the 10 replicas will be scheduled to healthy worker nodes. In the background, Deployment and ReplicaSet controllers are watching the state of play and eager to perform their reconciliation magic.

Inspecting Deployments

You can use the normal `kubectl get` and `kubectl describe` commands to see details of Deployments and ReplicaSets.

```
$ kubectl get deploy hello-deploy
NAME          DESIRED   CURRENT  UP-TO-DATE  AVAILABLE   AGE
hello-deploy  10        10       10          10          24s

$ kubectl describe deploy hello-deploy
Name:                   hello-deploy
Namespace:              default
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=hello-world
Replicas:               10 desired | 10 updated | 10 total | 10 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        10
RollingUpdateStrategy:  1 max unavailable, 1 max surge
Pod Template:
  Labels:  app=hello-world
  Containers:
   hello-pod:
    Image:        nigelpoulton/k8sbook:1.0
    Port:         8080/TCP
<SNIP>
OldReplicaSets:  <none>
NewReplicaSet:   hello-deploy-65cbc9474c (10/10 replicas created)
<Snip>
```

The command outputs have been trimmed for readability, but take a minute to look at them as they contain a lot of important information.

As mentioned earlier, Deployments automatically create associated ReplicaSets. Check this with the following command.

```
$ kubectl get rs
NAME                     DESIRED   CURRENT  READY   AGE
hello-deploy-65cbc9474c  10        10       10      1m
```

Right now you only have one ReplicaSet. This is because you've only performed an initial rollout. However, you can see the name of the ReplicaSet matches the name of the Deployment plus a hash on the end. This is a crypto-hash of the Pod template section of the Deployment manifest (everything below `.spec.template`). You'll see this

shortly, but making changes to the Pod template section initiates a rollout and a new ReplicaSet with a hash of the updated Pod template.

You can get more detailed information about the ReplicaSet with the usual `kubectl describe` command. Your ReplicaSet will have a different name

```
$ kubectl get rs hello-deploy-65cbc9474c
Name:           hello-deploy-65cbc9474c
Namespace:      default
Selector:       app=hello-world,pod-template-hash=65cbc9474c
Labels:         app=hello-world
                pod-template-hash=65cbc9474c
Annotations:    deployment.kubernetes.io/desired-replicas: 10
                deployment.kubernetes.io/max-replicas: 11
                deployment.kubernetes.io/revision: 1
Controlled By:  Deployment/hello-deploy
Replicas:       10 current / 10 desired
Pods Status:    10 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app=hello-world
           pod-template-hash=65cbc9474c
  Containers:
   hello-pod:
    Image:        nigelpoulton/k8sbook:1.0
    Port:         8080/TCP
<Snip>
```

Notice how the output is similar to the Deployment output. This is because the config of the ReplicaSet is dictated by the Deployment that created it. The status (observed state) of the ReplicaSet is also rolled up into the Deployment status.

As with the Deployment output, the ReplicaSet output tells us a lot about it and how it connects to Pods and its governing Deployment.

Accessing the app

As things stand, you've got 10 replicas of a web app running. In order to access it from a stable name or IP address, or even from outside the cluster, you need a Kubernetes *Service* object. We'll discuss these in detail in the next chapter, but for now it's enough to know they provide networking for a set of Pods.

The following YAML defines a Service that works with the Pod replicas previously deployed. It's included in the "deployments" folder of the book's GitHub repo called `svc.yml`.

```
apiVersion: v1
kind: Service
metadata:
  name: hello-svc
  labels:
    app: hello-world
spec:
  type: NodePort
  ports:
  - port: 8080
    nodePort: 30001
    protocol: TCP
  selector:
    app: hello-world
```

Deploy it with the following command. Be sure to run the command from the `deployments` directory.

```
$ kubectl apply -f svc.yml
service/hello-svc created
```

Now the Service is deployed, you can access the app by hitting any of the cluster nodes on port `30001`

If you're running Docker Desktop you should be able to use `localhost:30001`. Unfortunately, the current versions of Docker Desktop for Mac contain a bug and this doesn't work. It may be fixed in your version.

If you're running your cluster in the cloud, you need to hit the public IP or public DNS name of one of your cluster nodes on `30001`.

If you're using Minikube, get your Minikube IP (`$ minikube ip`) and append port `30001`.

Figure 6.5 shows the Service being accessed from outside the cluster via a node called `node1` on port `30001`. It assumes `node1` is resolvable via DNS, and port `30001` is allowed on any intervening firewalls.

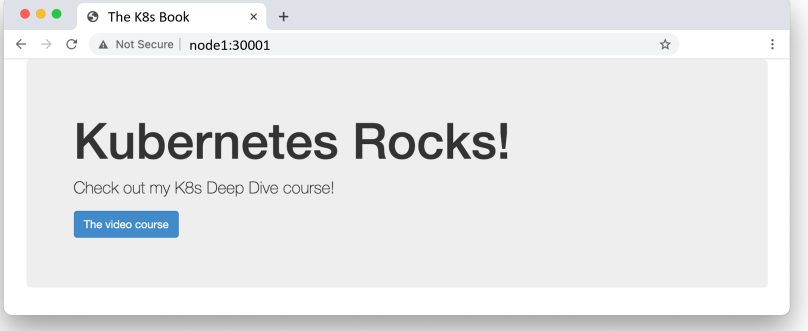

Figure 6.5

Perform scaling operations

Manually scaling the number of replicas in a Deployment is easy. You can do it imperatively with the `kubectl scale` command, or declaratively by updating the YAML file and re-posting to the API server. You'll do it both ways, but the preferred way is the declarative way.

Verify the current number of replicas.

```
$ kubectl get deploy hello-deploy
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
hello-deploy   10/10   10           10          27m
```

Run the following command to scale down to 5 and verify the operation.

```
$ kubectl scale deploy hello-deploy --replicas 5
deployment.apps/hello-deploy scaled

$ kubectl get deploy hello-deploy
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
hello-deploy   5/5     5            5           27m
```

You've successfully scaled the Deployment, but there's a potential problem. The state of your environment no longer matches what is in your declarative manifest – the cluster has 5 replicas but the Deployment YAML still defines 10. This can cause issues in the future if you use the YAML file to update other properties of the Deployment. For example, if you update the image version in the YAML file and post it to the API server, you'll also increase the replica count back to 10. For this reason, you should always keep your YAML manifests in sync with your live environment.

Let's re-post the YAML file and take the replica count back to 10. Editing the YAML file and posting it to Kubernetes is the preferred way to manually scale apps.

You can edit the YAML file and set a different number of replicas, but the examples later in the chapter will assume you have 10.

```
$ kubectl apply -f deploy.yml
deployment.apps/hello-deploy configured

$ kubectl get deploy hello-deploy
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
hello-deploy   10/10   10           10          29m
```

You may have noticed the scaling operations were almost instantaneous. This is different to rolling updates that you're going to see next.

Kubernetes also has autoscalers that scale Pods and infrastructure based on resource demand.

Perform a rolling update

In this section, you'll see how to perform a rolling update on the app already deployed. We'll assume the new version of the app has already been created and containerized as an image with the `2.0` tag. All that's left to do is perform the rollout. To simplify the process and keep the focus on Kubernetes, we'll ignore real-world CI/CD workflows and version control tools.

Before continuing, it's vital you understand that update operations are ***replacement operations***. When you "update" a Pod, you're actually terminating it and replacing it with a brand new one. Pods are designed as *immutable objects,* so you **never** change or update existing ones.

Ok, let's crack on.

The first step is to update the image version in the Deployment's resource manifest. The initial release of the app is using the `nigelpoulton/k8sbook:1.0` image. Update that to reference the newer `nigelpoulton/k8sbook:2.0` image and save your changes. This ensures next time the manifest is posted to the API server, all Pods managed by the Deployment will be replaced with new ones running the new `2.0` image.

The following trimmed output shows the updated file. The only change is to the container image line in the Pod template section.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  <Snip>
  template:
    <Snip>
    spec:
      containers:
      - name: hello-pod
        image: nigelpoulton/k8sbook:2.0   <<-- The only line that changed
        ports:
        - containerPort: 8080
```

Before posting it to Kubernetes, let's look at the settings governing how the rollout will work.

> **Terminology:** We often use the terms *update, rollout,* and *release* to mean the same thing – issuing a new version of an app.

The `.spec` section of the manifest contains all the settings governing how updates will be performed.

```
<Snip>
revisionHistoryLimit: 5
progressDeadlineSeconds: 300
minReadySeconds: 10
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
<Snip>
```

`revisionHistoryLimit` tells Kubernetes to keep the configs of the previous 5 releases. This means the previous 5 ReplicaSet objects will be kept and you can rollback to any of them.

`progressDeadlineSeconds` governs how long Kubernetes waits for each new Pod replica to start, before considering the rollout to have stalled. This config gives each Pod replica its own 5 minute window to come up.

`.spec.minReadySeconds` throttles the rate at which replicas are replaced. The one in the example tells Kubernetes that any new replica must be up and running for 10 seconds, without any issues, before it's allowed to update/replace the next one in sequence. Longer waits give you a chance to spot problems and avoid updating all replicas to a dodgy version. In the real world, you should make the value large enough to trap common failures.

There is also a nested `.spec.strategy` map telling Kubernetes you want this Deployment to:

- Update using the `RollingUpdate` strategy
- Never have more than one Pod below desired state (`maxUnavailable: 1`)
- Never have more than one Pod above desired state (`maxSurge: 1`)

As the desired state of the app requests 10 replicas, `maxSurge: 1` means you'll never have more than 11 replicas during the update process, and `maxUnavailable: 1` means you'll never have less than 9. The net result is a rollout that updates two Pods at a time (the delta between 9 and 11 is 2).

What you've seen so far is great, but we haven't explained how Kubernetes knows which Pods to terminate and replace when performing the update.

The answer to this is simple but vital… label selectors!

If you look closely at the `deploy.yml` file, you'll see the Deployment spec has a selector block. This is a list of labels the Deployment controller looks for when finding Pods to update during rollout operations. In this example, it's looking for Pods with the `app=hello-world` label, and the Pod template will create Pods with that label. Net result… the Pods created by this Deployment will be managed by it.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  selector:                    <<=== The Deployment will manage all
    matchLabels:               <<=== replicas on the cluster with
      app: hello-world         <<=== this label
      <Snip>
  template:
    metadata:
      labels:
        app: hello-world       <<=== Matches the label selector
<Snip>
```

The Deployment's label selector is immutable, so you can't change it once it's deployed.

With the updated manifest ready and saved, you can initiate the update by re-posting it to the API server. Be sure to add the `--record` flag this time as we'll reference it later.

```
$ kubectl apply -f deploy.yml --record
deployment.apps/hello-deploy configured
```

The update may take some time to complete. This is because it's incrementing two Pods at a time, pulling the new image on each node, starting the new Pods, and then waiting 10 seconds before moving on to the next two.

You can monitor the progress with `kubectl rollout status`.

```
$ kubectl rollout status deployment hello-deploy
Waiting for rollout to finish: 4 out of 10 new replicas...
Waiting for rollout to finish: 4 out of 10 new replicas...
Waiting for rollout to finish: 6 out of 10 new replicas...
^C
```

If you press `Ctrl+C` to stop watching the progress, you can run `kubectl get deploy` commands while the update is in process. This lets you see the effect of some of the update-related settings in the manifest. For example, the following command shows that 5 of the replicas have been updated and you currently have 11. 11 is 1 more than the desired state of 10. This is a result of the `maxSurge=1` value in the manifest.

```
$ kubectl get deploy
NAME           DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
hello-deploy   10        11        5            11          31m
```

Pausing and resuming rollouts

It's possible to pause and resume a rollout using `kubectl`.

If the rollout is still in progress, you can pause it with the following command.

```
$ kubectl rollout pause deploy hello-deploy
deployment.apps/hello-deploy paused
```

Running a `kubectl describe` provides some interesting info.

```
$ kubectl describe deploy hello-deploy
Name:                   hello-deploy
Annotations:            deployment.kubernetes.io/revision: 2
Selector:               app=hello-world
Replicas:               10 desired | 4 updated | 11 total | 11 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        10
RollingUpdateStrategy:  1 max unavailable, 1 max surge
<Snip>
Conditions:
  Type           Status   Reason
  ----           ------   ------
  Available      True     MinimumReplicasAvailable
  Progressing    Unknown  DeploymentPaused
OldReplicaSets:  hello-deploy-85fd664fff (7/7 replicas created)
NewReplicaSet:   hello-deploy-5445f6dcbb (4/4 replicas created)
```

The `deployment.kubernetes.io` annotation shows the object is on revision 2 (revision 1 was the initial rollout and this update is revision 2). `Replicas` shows the rollout is incomplete. The third line form the bottom shows the Deployment condition as "progressing" but paused. Finally, you can see the ReplicaSet for the initial release is wound down to 7 replicas and the one for the new release is up to 4.

If a scale-up operation occurs during a rollout, the additional replicas will be balanced across both ReplicaSets. In this example, if the Deployment was increased to 20 by adding 10 new replicas, Kubernetes would assign 7 of the new replicas to the old ReplicaSet and 4 to the new. This happens even if the rollout is not paused.

You can resume the rollout with the following command.

```
$ kubectl rollout resume deploy hello-deploy
deployment.apps/hello-deploy resumed
```

Once it completes, you can verify with `kubectl get deploy`.

```
$ kubectl get deploy hello-deploy
NAME          DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
hello-deploy  10        10        10           10          39m
```

The output shows the rollout as complete – 10 Pods are up-to-date and available.

If you've been following along with the examples, you'll be able to hit `refresh` in your browser and see the updated app (Figure 6.6). The old version displayed "Kubernetes Rocks!", the new one displays "The Kubernetes Book!!!".

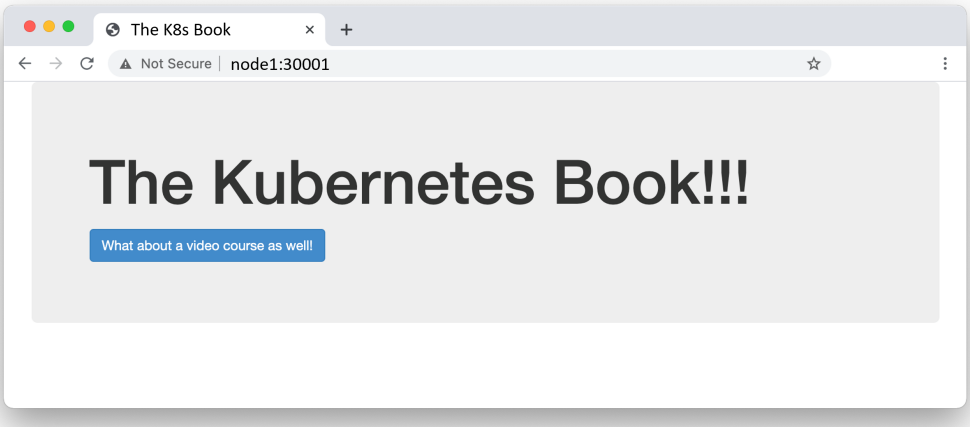

Figure 6.6

Perform a rollback

A moment ago, you used `kubectl apply` to perform a rolling update on the "hello-deploy" Deployment. You used the `--record` flag so Kubernetes would maintain a documented revision history. The following command shows the Deployment with two revisions.

```
$ kubectl rollout history deployment hello-deploy
deployment.apps/hello-deploy
REVISION  CHANGE-CAUSE
1         <none>
2         kubectl apply --filename-deploy.yml --record=true
```

Revision `1` was the initial deployment that used the `1.0` image tag. Revision `2` is the rolling update you just performed. You can see the command used to invoke the update has been recorded in the object's history. This is only there because you used the `--record` flag when invoking the update. This might be a good reason for you to use the `--record` flag.

You know that rolling updates create new ReplicaSets, and that old ReplicaSets aren't deleted. The fact the old ones still exist makes them ideal for executing rollbacks, which are the same as updates, just in reverse.

The following commands show the two ReplicaSet objects. The second command shows the config of the old one and that it still references the old image version. The output is trimmed to fit the book.

```
$ kubectl get rs
NAME                      DESIRED   CURRENT   READY   AGE
hello-deploy-65cbc9474c   0         0         0       42m
hello-deploy-6f8677b5b    10        10        10      5m

$ kubectl describe rs hello-deploy-65cbc9474c
Name:           hello-deploy-65cbc9474c
Namespace:      default
Selector:       app=hello-world,pod-template-hash=65cbc9474c
Labels:         app=hello-world
                pod-template-hash=65cbc9474c
Annotations:    deployment.kubernetes.io/desired-replicas: 10
                deployment.kubernetes.io/max-replicas: 11
                deployment.kubernetes.io/revision: 1
                kubernetes.io/change-cause: kubectl apply --filename=deploy.yml --record=true
Controlled By:  Deployment/hello-deploy
Replicas:       0 current / 0 desired
Pods Status:    0 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Containers:
   hello-pod:
    Image:        nigelpoulton/k8sbook:1.0
    Port:         8080/TCP
```

The line you're interested in is the one shown second-from-last in the book, as it shows the old image version. Therefore, flipping the Deployment back to this ReplicaSet will automatically replace all Pods with the original version.

> **Note:** Don't get confused when I refer to the rollback as an "update". That's exactly what it is. A rollback follows exactly the same logic and rules as a rollout – terminate Pods with the current image and replace them with Pods running the new image. Just in the case of a rollback, the "new" image is actually an "older" one.

The following example uses `kubectl rollout` to revert the application to revision 1. This is an imperative operation and not recommended. However, it's convenient for quick rollbacks, just make sure you remember to update your source YAML files to reflect the changes you make.

```
$ kubectl rollout undo deployment hello-deploy --to-revision=1
deployment.apps "hello-deploy" rolled back
```

Although it might look like the operation is instantaneous, it isn't. Like we said before, rollbacks follow the same rules defined in the `strategy` block of the Deployment object. In this example, that'll be `minReadySeconds: 10`, `maxUnavailable: 1`, and `maxSurge: 1`. You can verify this and track the progress with the following `kubectl get deploy` and `kubectl rollout` commands.

```
$ kubectl get deploy hello-deploy
NAME          DESIRED  CURRNET  UP-TO-DATE  AVAILABE  AGE
hello-deploy  10       11       4           9         45m

$ kubectl rollout status deployment hello-deploy
Waiting for rollout to finish: 6 out of 10 new replicas have been updated...
Waiting for rollout to finish: 7 out of 10 new replicas have been updated...
Waiting for rollout to finish: 8 out of 10 new replicas have been updated...
Waiting for rollout to finish: 1 old replicas are pending termination...
Waiting for rollout to finish: 9 of 10 updated replicas are available...
^C
```

Congratulations. You've performed a rolling update and a successful rollback.

Just a quick reminder. The rollback operation you performed was an imperative one. This means the current state of the cluster no longer matches your source YAML files – the latest version of the YAML file lists the `2.0` image, but you rolled the cluster back to `1.0`. This is a fundamental flaw with the imperative approach. In the real world, following a rollback operation like this, you should manually update your source YAML files to reflect the changes invoked by the rollback.

Rollouts and labels

You've already seen that Deployments and ReplicaSets use labels and selectors to find Pods they own.

In earlier versions of Kubernetes, it was possible for Deployments to take over management of existing static Pods if they had the same labels. However, recent versions use the system-generated `pod-template-hash` label so only Pods created by the Deployment/ReplicaSet will be managed.

Assume a quick example. You already have 5 Pods on a cluster with the `app=front-end` label. At a later date, you create a Deployment that requests 10 Pods with the same `app=front-end` label. Older versions Kubernetes would notice there were already 5 Pods with that label and only create 5 new ones, and the Deployment/ReplicaSet would manage all 10. However, newer versions of Kubernetes tag all Pods created by a Deployment/ReplicaSet with the `pod-template-hash` label. This stops higher-level controllers seizing ownership of existing static Pods.

The following extremely snipped outputs show how this label connects everything.

```
$ kubectl describe deploy hello-deploy
Name:      hello-deploy
<Snip>
NewReplicaSet:   hello-deploy-5445f6dcbb

$ kubectl describe rs hello-deploy-5445f6dcbb
Name:           hello-deploy-5445f6dcbb
<Snip>>
Selector:       app=hello-world,pod-template-hash=5445f6dcbb

$ kubectl get pods --show-labels
NAME                        READY   STATUS    LABELS
hello-deploy-5445f6dcbb..   1/1     Running   app=hello-world,pod-template-hash=5445f6dcbb
hello-deploy-5445f6dcbb..   1/1     Running   app=hello-world,pod-template-hash=5445f6dcbb
hello-deploy-5445f6dcbb..   1/1     Running   app=hello-world,pod-template-hash=5445f6dcbb
```

```
hello-deploy-5445f6dcbb..   1/1     Running   app=hello-world,pod-template-hash=5445f6dcbb
<Snip>
```

You shouldn't mess about with `pod-template-hash` label or selector.

Clean-up

Use `kubectl delete -f deploy.yml` and `kubectl delete -f svc.yml` to delete the Deployment and Service created in the examples.

Chapter summary

In this chapter, you learned that *Deployments* are a great way to manage stateless apps on Kubernetes. They augment Pods with *self-healing, scalability, rolling updates,* and *rollbacks.*

Behind-the-scenes, Deployments use ReplicaSets to do most of the work with Pods – it's actually a ReplicaSet that creates, terminates, and otherwise manages Pods, but the Deployment tells the ReplicaSet what to do.

Like Pods, Deployments are objects in the Kubernetes API and you should work with them declaratively. They're defined in the `apps/v1` workloads API sub-group and implement a controller architecture running as a reconciliation loop on the control plane.

When you perform updates with the `kubectl apply` command, previous versions of ReplicaSets get wound down, but they stick around making it easy to perform rollbacks.